

Application Development

Proposal V2

Submitted by: Lord Raizen I. Espinosa

1. Proposal, version 2

App name

Coffee App

The problem, in one sentence

My coffee is inconsistent because I struggle to keep track of my recipes and the variables that can affect it like grind size, roast type, and brewing time.

Who is this for

This is for me because I am a coffee addict who happens to brew with 3 brewing methods: the moka pot, the french press, and cold brew. I am also supported by a Timemore C3ESP hand grinder and a Timemore Black Mirror 3 coffee scale. I also do not make any attempt in recording any of my recipes or variables, leading to inconsistent coffee.

Core features (MVP), revised

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest estimate
1	Ratio Calculator	keep	2 TextFields for water and coffee grams, a StatefulWidget recalculating on every keystroke, Container or badge for the live strength label	12 hours
2	Grind Setting	keep (stick to one grinder for the release version)	2 DropdownButtonFormFields for brew type and roast level, a Dart Map lookup table, Container for the results	12 hours
3	Stopwatch	keep	StatefulWidget with setState, Dart's Timer.periodic for the function, 3 ElevatedButtons for start, stop, and reset	24 hours
4	Notes	keep	ListView.builder for saved recipes, showDialog or new-screen form with TextFields, Navigator.push for detail view, hive_ce box read/write	24 hours

Estimate time: 3 days in total, probably more like a few weeks in practice. Mostly due to me not being good at Dart and the definite struggle that will be had in matching the UI designed by me. It does not help that I tend to lose focus a lot with work and I still have other subjects I need time to tend to.

Stretch goals

1. Support for additional grinders, especially the ones that were removed.
2. Have Notes gather the user inputs and automatically fill it in a recipe template
3. Support for other units of measurements apart from grams (although, you should use grams to begin with, especially if owning a coffee scale)

NEW: How my app saves data

- **The question from page 1:** if two different people install my app, should they see the same data? No. Each instance of this app is meant to be the personal brew history of a person. Sharing is not needed
- **Roughly how many records** Probably 2=5 recipes. I like to experiment a lot, especially on the moka pot.
- **My choice:** hive_ce
- **Why this one and not the others** I went with Hive because it works on all Flutter platforms, especially the browser, where I will run and test my app, and will handle my recipe data efficiently. It will not rewrite the entire dataset like shared_preferences and it does what I need without being too complex.
- **What I save, concretely:** A recipes Hive box with one entry per recipe as a plain map with these fields: brewMethod, coffeeGrams, waterGrams, ratio, strengthLabel, roastLevel, grindSetting, brewTimeSeconds, tasteNotes, tempCelsius, and dateCreated. No TypeAdapter or code generation is required because it is stored as a map, not a custom class. The grinder settings data is separate: it is a fixed reference data, not user data, so it stays as a Dart Map constant in code instead of going into the box.
- **Have I tried it yet?** Not yet. I will do it after the exams.

NEW: One thing I want to add that the course did not teach

Nothing. I want to finish what I have. The one concern I do have is modularity. I want to ensure my app is still modular so I can easily add new things like grinders and brew methods without doing a total rewrite. I see this as a constraint in my app.

NEW: How my project runs when someone else opens it

- I am keeping the `device_preview` wrapper: Yes.
- My app runs in a browser with `flutter run -d web-server`, start to finish, with every screen reachable: Not yet because I have not implemented anything. It is why I chose Hive for storage. It works in the browser so the data layer will not block it once screens are up.
- Anything that needs real hardware degrades to sample data instead of crashing: Not applicable for the app will not be using things like cameras, sensors, nor microphones. Simply inputs and a stopwatch.
- My project will live in a **public repository in my own GitHub account**, not the course org. Does anything I plan to build need an API key, a password, or real personal data? None is needed. Hive is locally stored on the device and browser without the need for a server, account, nor API key. No need for .env files, and repo secrets. No private information can be leaked then, unless you really want someone's secret coffee recipe (Are you going into coffee, Plankton?).

Data the app remembers

Thing	Fields	Where it is saved
Recipe	brewMethod, coffeeGrams, waterGrams, ratio, strengthLabel, roastLevel, grindSetting, brewTimeSeconds, tasteNotes, tempCelsius, dateCreated	hive_ce local box, recipes
Grinder profile (Timemore C3ESP only for MVP)	brewMethod, roastLevel, clickSetting	Dart Map constant in code (static reference data, not user data)

Screens

1. Home
2. Calculator
3. Roast and Grind
4. Timer
5. Notes

Risks, revised

- **The risk I named last time:** The risk I had was whether or not I was biting off more than I can chew by having 3 grinders to research grind settings for 3 brew types and 2 types of roast levels.
- **A new risk I did not see before:** It is my own inability to be focused and to be good with Dart and UI design. It is part of why my estimated times are so high. A risk I now believe in is probably not having the adequate time to finish this in a timely manner.
- **For each: what is my first step to reduce it, and when?** I suppose I should lock in and do proper time management. If I do that, it should make things easier for me and allow me to beat the high estimations I gave myself. Definitely practice more Dart and Flutter. As to when, before and after exams, especially before. Why? I will be reviewing Dart and Flutter anyway, which will also benefit my ability in developing my app.

What changed, and why

The graded part. One row per change, with a reason that refers to something you actually did.

Section	Prelim said	Now says	Why it changed
Core features	Timer and Stopwatch, meaning a countdown and stopwatch function	Just the stopwatch with start, stop, and reset, but no duration-set input for a countdown	It matches what was in the wireframe and the first mockup, which never included a duration field for a timer to begin with. This legitimizes a decision that had already been made implicitly when the screens were designed and addresses a doubt I had with the timer adding complexity.
Grinder Selection	I wanted the Comandante C40, Kingrinder K2, and Timemore C3ESP at first	Decided to stick with just the C3ESP for the release version as a minimum viable product. The other 2 and potentially other grinders are a stretch goal now.	I realized I would need to dedicate a lot of my time in research alone because each grinder means grind settings for 3 brew types alongside 2 roast level per type. I decided to just stick with the grinder I actually own because I already know how it works, saving me time.
Data Storage	Although not mentioned in prelim, I first considered sqlite due to my familiarity with SQL.	Now it's hive_ce	sqlite does not work on browsers unlike hive_ce, which is an issue because I will test on the browser. Plus, no loss of the relational structure because my recipe is a flat table anyway.